

📊 Linear Regression — Complete Guide (Urdu + English)

Asal maqsad: Yeh guide aap ko Linear Regression ka poora concept — start se advanced tak — simple zaban mein, graphs aur code ke sath samjhaye gi.

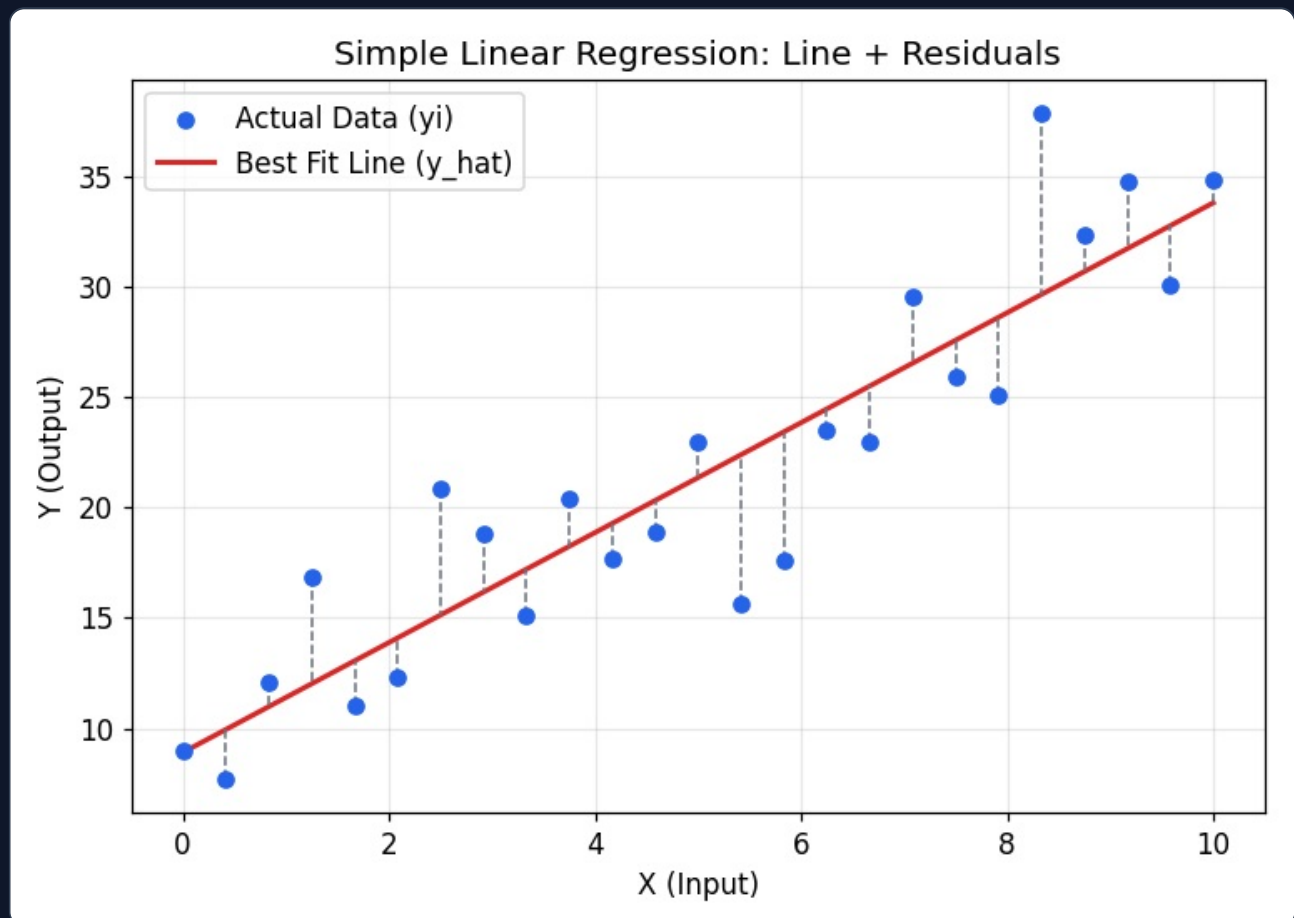
1 Simple Linear Regression (Ek Input)

Jab hamare paas sirf **ek input (X)** aur ek **output (Y)** ho, to hum ek seedhi line fit karte hain jo X aur Y ke darmiyan relationship batati hai.

$$\hat{y} = m \cdot x + b$$

$\hat{y}$ = predicted output | m = slope (line kitni tez upar/neeche jati hai) | b = intercept (line jahan Y-axis ko cut karti hai)

Example: X = Ghar ka size (sq ft), Y = Ghar ki price. Jitna bara size, utni zyada price — is relationship ko line se model kiya jata hai.



2 Multiple Linear Regression (Multi Input)

Jab ek se zyada inputs hon ($X_1, X_2, X_3, \dots$) to hum unko combine karke output predict karte hain. Formula thora expand ho jata hai:

$$\hat{y} = b + m_1 \cdot X_1 + m_2 \cdot X_2 + m_3 \cdot X_3 + \dots + m_n \cdot X_n$$

Example: Ghar ki price predict karni ho to inputs: size, bedrooms, location score, age of house — sab milkar price predict karenge. Har input ka apna weight (m) hota hai jo batata hai wo output pe kitna asar dalta hai.

Code (Python — sklearn):

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Multi input example
X = np.array([[1200, 3, 8], [1500, 4, 6], [800, 2, 9], [2000, 5, 5]]) # size, bedrooms, loc
y = np.array([250000, 310000, 180000, 400000]) # price

model = LinearRegression()
model.fit(X, y)

print("Weights (m):", model.coef_)
print("Intercept (b):", model.intercept_)
print("Prediction:", model.predict([[1300, 3, 7]]))
```

3 Residual (Error)

Residual matlab: **actual value aur predicted value ka farq**. Yeh batata hai model kitna galat hai us particular point pe.

$$\text{residual } (e_i) = y_i - \hat{y}_i$$

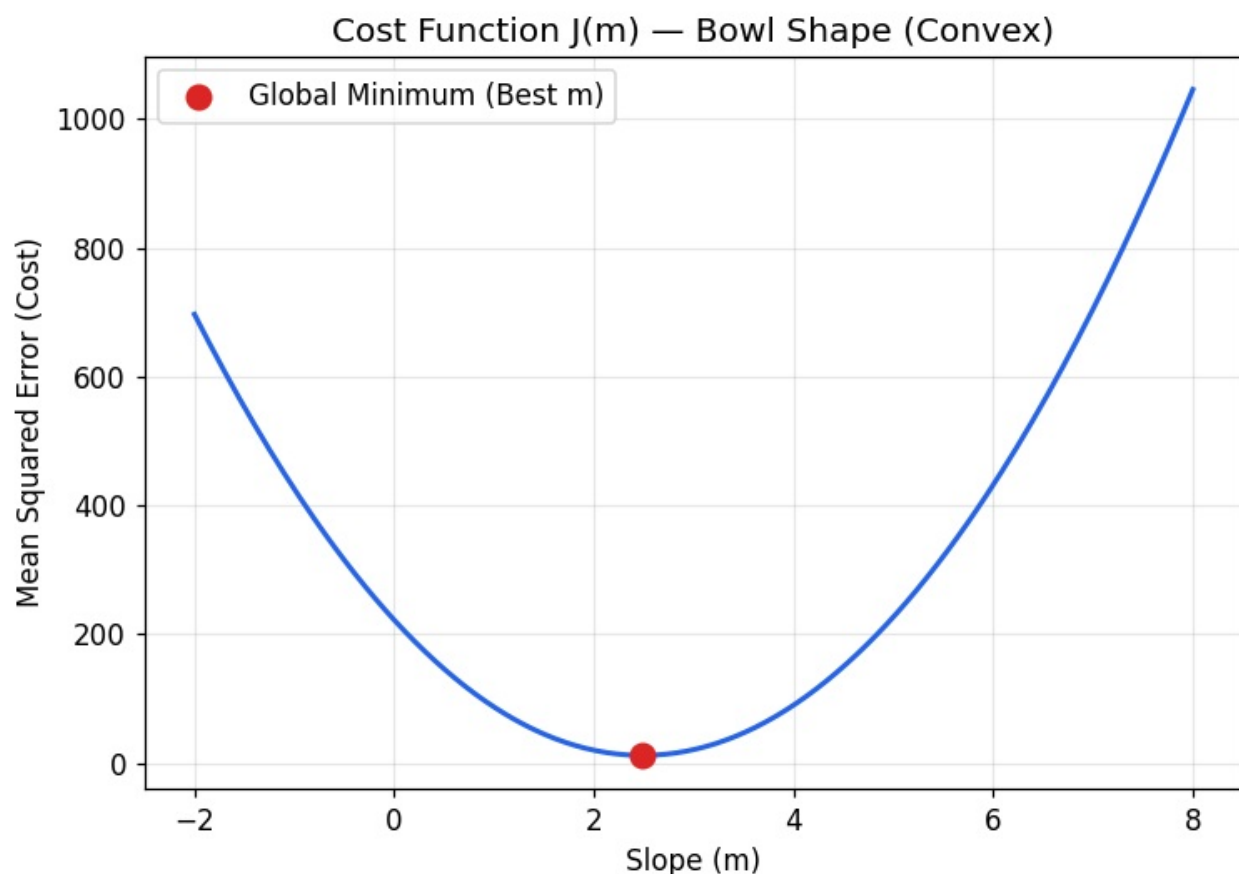
Agar residual zero ke qareeb ho to matlab prediction accha hai. Agar residual bara ho (positive ya negative) to matlab model us point ko theek predict nahi kar raha.

4 Mean Squared Error (MSE) — Cost Function

Hum sab residuals ko square karke unka average nikalte hain — isko **Cost Function J** bhi kehte hain. Square isliye karte hain taake negative aur positive errors cancel na hon, aur bare errors ko zyada punish kiya jaye.

$$MSE = J(m, b) = (1/n) \cdot \sum (y_i - \hat{y}_i)^2$$

Model ka poora goal hai: **is Cost Function ko minimum karna** — jitna kam MSE, utna acha model.



Cost function ek "bowl" (U-shape) hoti hai — sab se neeche wala point = best parameters

5 Gradient Descent

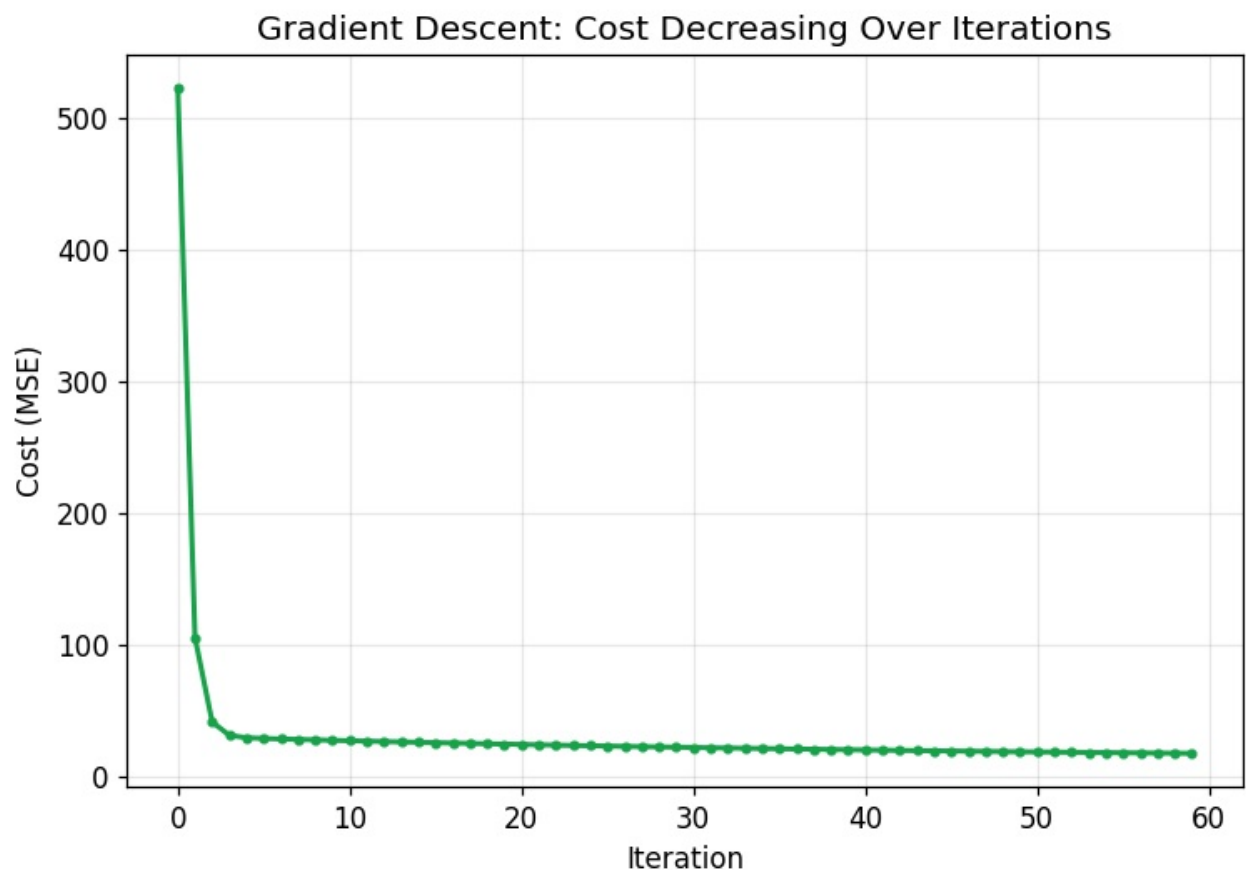
Gradient Descent ek algorithm hai jo Cost Function ke minimum point (bowl ke tal) tak pohanchne ke liye **chote chote steps** leta hai, har step pe slope (m) aur intercept (b) ko update karta hai.

$$m = m - \alpha \cdot \frac{\partial J}{\partial m}$$
$$b = b - \alpha \cdot \frac{\partial J}{\partial b}$$

α (alpha) = Learning Rate — yeh decide karta hai step kitna bara ho

Derivatives (gradients) yeh formula se nikalte hain:

$$\frac{\partial J}{\partial m} = -(2/n) \cdot \sum x_i(y_i - \hat{y}_i)$$
$$\frac{\partial J}{\partial b} = -(2/n) \cdot \sum (y_i - \hat{y}_i)$$



Har iteration ke sath cost neeche jati hai — jab tak minimum na aa jaye

Code (Gradient Descent from scratch):

```
import numpy as np

def gradient_descent(x, y, lr=0.01, iterations=1000):
    m, b = 0, 0
    n = len(x)
    for i in range(iterations):
        y_pred = m*x + b
        dm = (-2/n) * sum(x*(y - y_pred))
        db = (-2/n) * sum(y - y_pred)
        m = m - lr*dm
        b = b - lr*db
    return m, b

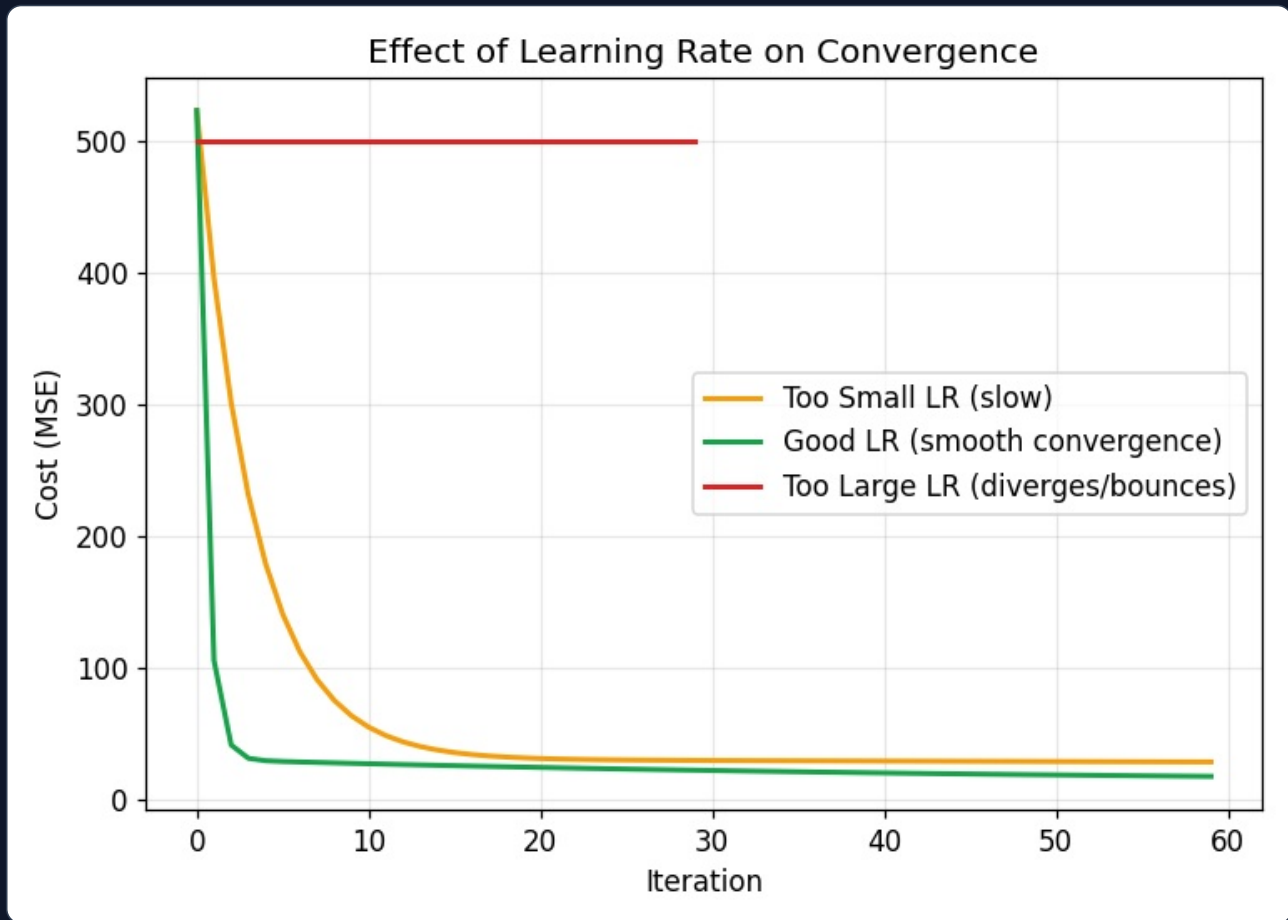
m, b = gradient_descent(np.array([1,2,3,4]), np.array([2,4,6,8]), lr=0.01, iterations=1000)
print(f"m={m:.2f}, b={b:.2f}")
```

6 Learning Rate (α)

Learning rate decide karta hai gradient descent kitni jaldi ya aahista seekhta hai:

Learning Rate	Effect
Bohat chota (0.0001)	Training bohat slow hogi, waqt zyada lagega

Sahi (0.01 – 0.1 range, tune karna parta hai)	Smooth aur fast convergence
Bohat bara (1, 10)	Model minimum ko "overshoot" kar jata hai, kabhi converge hi nahi hota (diverge)



7 Train-Test Split

Model ko sirf ek hi data pe train aur test nahi karte, warna hum yeh nahi jaan payenge ke model naye (unseen) data pe kaisa perform karega. Isliye data ko do parts mein baant dete hain:

- **Training set** (usually 70-80%) — model isi se seekhta hai
- **Testing set** (usually 20-30%) — model ki performance yahan check karte hain

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

8 Model Evaluation Metrics

Metric	Formula	Kya batati hai
--------	---------	----------------

MAE (Mean Absolute Error)	$(1/n)\sum y_i - \hat{y}_i $	Average error (bina square kiye)
MSE (Mean Squared Error)	$(1/n)\sum (y_i - \hat{y}_i)^2$	Average squared error, bare errors ko zyada punish karta hai
RMSE	$\sqrt{\text{MSE}}$	MSE ka square root — same units mein hota hai jaisa Y
R ² Score	$1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$	Model kitna % variance explain karta hai (0 se 1, jitna zyada utna acha)

R² (R-Squared)

$$R^2 = 1 - (\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2)$$

R² = 1 matlab perfect model. R² = 0 matlab model sirf average predict kar raha hai, kuch seekha nahi.

Adjusted R²

Problem yeh hai ke R² hamesha bara hota rehta hai jab hum zyada features add karte jayen — chahay wo features useful hon ya na hon. Isliye **Adjusted R²** use karte hain jo useless features ke liye penalty deta hai.

$$\text{Adjusted } R^2 = 1 - [(1-R^2)(n-1) / (n-k-1)]$$

n = total samples | k = number of input features

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

n, k = X_test.shape[0], X_test.shape[1]
adj_r2 = 1 - (1-r2)*(n-1)/(n-k-1)

print(f"MAE={mae:.2f}, RMSE={rmse:.2f}, R2={r2:.3f}, Adjusted R2={adj_r2:.3f}")
```

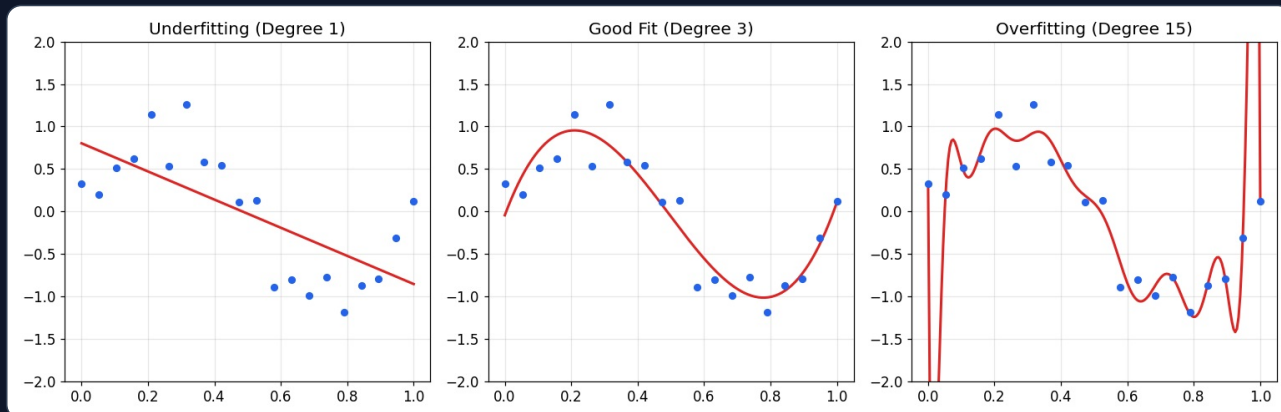
9 Overfitting, Underfitting & Best Fit

Underfitting: Model bohat simple hai, data ka pattern seekh hi nahi paata (training aur testing dono pe kharab perform karta hai).

Best Fit: Model sahi complexity ka hai — general pattern seekhta hai, training aur test dono pe achi performance.

Overfitting: Model itna complex hai ke wo training data ka noise bhi "yaad" kar leta hai.

Training pe bohat acha lekin naye data pe kharab perform karta hai.



Ridge & Lasso Regression (Regularization)

Overfitting ko control karne ke liye hum Cost Function mein ek **penalty term** add karte hain — isko Regularization kehte hain.

Ridge Regression (L2 Regularization)

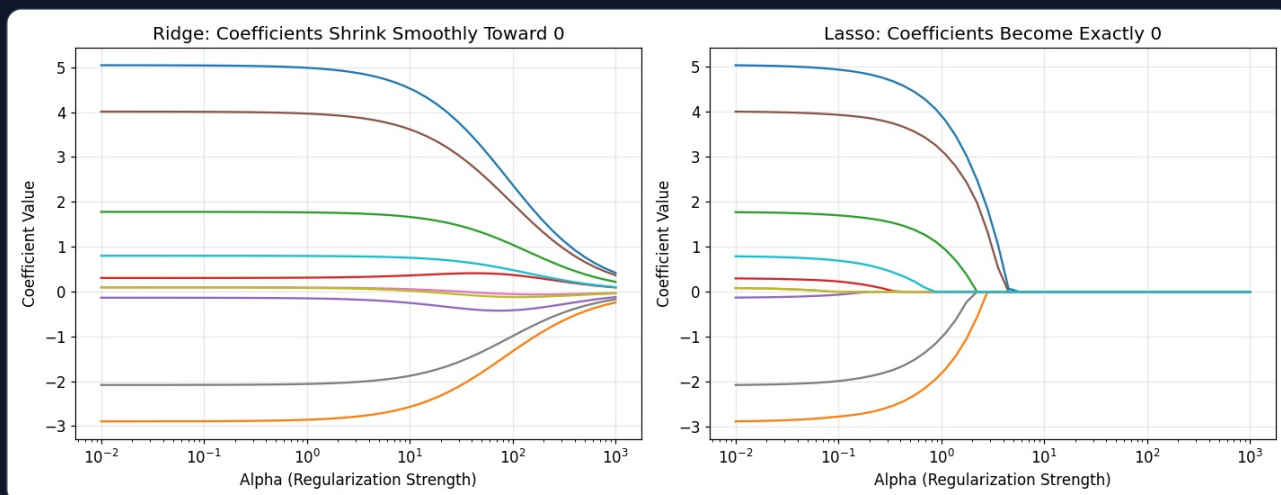
$$\text{Cost} = \text{MSE} + \lambda \cdot \sum m_i^2$$

Coefficients ko chota karta hai (shrink) lekin unhein bilkul zero nahi karta. Sab features rakhta hai, bas unka asar kam kar deta hai.

Lasso Regression (L1 Regularization)

$$\text{Cost} = \text{MSE} + \lambda \cdot \sum |m_i|$$

Lasso kuch coefficients ko bilkul **zero** kar deta hai — matlab yeh automatic feature selection bhi kar deta hai (useless features ko hata deta hai).



Ridge coefficients smoothly zero ke qareeb jate hain, Lasso unhe bilkul zero kar deta hai

```

from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1.0) # alpha = lambda (regularization strength)
ridge.fit(X_train, y_train)

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)

print("Ridge coefficients:", ridge.coef_)
print("Lasso coefficients:", lasso.coef_) # kuch coefficients 0.0 honge

```

Kab konsa use karein? Agar samajh nahi aata konse features important hain aur zyada features hain to **Lasso** use karein (feature selection ke liye). Agar sab features thora bohat useful hain to **Ridge** behtar hai.

11 StandardScaler (Feature Scaling)

Jab different features ka scale bohat different ho (jaise "size" 100-5000 range mein aur "bedrooms" 1-5 range mein), to model bara scale wale feature ko zyada important samajh sakta hai — jo sahi nahi hota. Isliye hum sab features ko **same scale** pe le aate hain.

$$z = (x - \text{mean}) / \text{std_deviation}$$

StandardScaler har feature ko is tarah transform karta hai ke uska **mean = 0** aur **standard deviation = 1** ho jata hai. Yeh Gradient Descent ko fast aur stable banata hai, aur Ridge/Lasso ke liye **zaroori** hai (warna penalty sirf bare-scale features pe zyada asar dalegi).

```

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit + transform on training data
X_test_scaled = scaler.transform(X_test)      # sirf transform on test data (fit dobara n

```

Important Rule: `fit_transform()` sirf training data pe karein. Test data pe sirf `transform()` use karein — warna "data leakage" ho jayega aur test results galat aayenge.

🔗 Complete End-to-End Example

```

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score

# 1. Data
X = np.random.randn(200, 5)
y = 3*X[:,0] - 2*X[:,1] + 5*X[:,2] + np.random.normal(0, 1, 200)

```

```
# 2. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Scaling
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# 4. Models
models = {
    "Linear": LinearRegression(),
    "Ridge": Ridge(alpha=1.0),
    "Lasso": Lasso(alpha=0.1)
}

for name, m in models.items():
    m.fit(X_train_s, y_train)
    pred = m.predict(X_test_s)
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    r2 = r2_score(y_test, pred)
    print(f"{name}: RMSE={rmse:.3f}, R2={r2:.3f}")
```

Quick Summary

Concept	One-line yaad rakhne wali baat
Linear Regression	Best straight line/plane fit karna data pe
Residual	Actual – Predicted (error ek point pe)
MSE / Cost Function	Average squared error — jise minimize karna hai
Gradient Descent	Cost minimize karne ka step-by-step tareeqa
Learning Rate	Step size — na bohat chota, na bohat bara
Train-Test Split	Unseen data pe performance check karne ke liye
R ² / Adjusted R ²	Model kitna variance explain karta hai (adjusted = penalty for extra features)
Overfitting	Training pe acha, test pe kharab (too complex)
Underfitting	Training aur test dono pe kharab (too simple)
Ridge (L2)	Coefficients shrink karta hai, zero nahi karta
Lasso (L1)	Coefficients ko zero bhi kar sakta hai (feature selection)
StandardScaler	Sab features ko same scale (mean=0, std=1) pe laata hai